

TradeSimulator v2.0 — Week 2 Complete Change Log

Change 1 — Multiple Pricing Algorithms (Strategy Pattern)

What changed	File	Exactly what was modified / added
Modified	PriceUpdateStrategies.java	Added ticker: String as first parameter to nextPrice() so each strategy can track per-ticker price history independently.
Modified	RandomWalkStrategy.java	Added @Component("randomWalk") annotation and updated nextPrice() signature to accept ticker param (ignored — random walk is ticker-independent).
Modified	MarketFeedServices.java	Added setStrategy(PricingUpdateStrategies), getStrategyName(); updated tick() to use Map.Entry so ticker is passed to nextPrice(); added @Qualifier("randomWalk") to constructor.
Modified	AppConfig.java	Removed @Bean PricingUpdateStrategies method — strategies now self-register via @Component so Spring resolves them by qualifier name.
Modified	RandomWalkStrategyTest.java	Updated all nextPrice() calls to include "AAPL" as first argument to match the updated interface signature.
New	MeanReversionStrategy.java	New strategy: tracks last 10 prices per ticker; pulls price toward moving average with strength proportional to distance from mean.
New	TrendFollowingStrategy.java	New strategy: tracks last 5 prices per ticker; continues price in direction of recent momentum; capped at +/-5% per tick.
New	StrategyController.java	GET /api/strategy returns current strategy name; POST /api/strategy switches strategy at runtime by calling marketFeedServices.setStrategy().
New	MeanReversionStrategyTest.java	4 tests: price reverts toward mean when above, reverts when below, tickers have independent histories, price always positive.
New	TrendFollowingStrategyTest.java	4 tests: price stays within 5% cap over 1000 iterations, independent ticker histories, price always positive, no momentum on first tick.
Frontend	index.html + app.js	Added strategy selector dropdown; fetchStrategy() syncs dropdown on load; changeStrategy() sends POST /api/strategy on dropdown change.

Change 2 — Multi-Channel Notifications (Decorator Pattern)

What changed	File	Exactly what was modified / added
Modified	NotificationDecorator.java	Added setWrapped(NotificationService) method so NotificationController can rewire the decorator chain at runtime when user changes channel preferences.
Modified	TradingService.java	Changed notificationService field from final to non-final; added setNotificationService() method; now gets notification service from active user via UserService.
Modified	ConsoleNotifier.java	Added @Primary annotation so Spring knows to inject ConsoleNotifier as the default NotificationService when multiple implementations exist.
Modified	AppConfig.java	Removed @Bean NotificationService method — ConsoleNotifier now self-registers via @Component @Primary.
Modified	NotificationServiceTest.java	Added 5 new tests: email decorator delegates to wrapped, SMS decorator delegates, dashboard increments badge and delegates, full Email+SMS chain, all four channels chain.
New	EmailDecorator.java	Decorator that logs formatted mock email (To, Subject, Body) then calls wrapped.notify() to continue the chain.
New	SmsDecorator.java	Decorator that logs a short mock SMS message then calls wrapped.notify() to continue the chain.
New	DashboardDecorator.java	Decorator that increments an AtomicInteger badge counter (thread-safe) then calls wrapped.notify(); exposes getBadgeCount() and resetBadgeCount().
New	NotificationController.java	POST /api/notifications/channels builds decorator chain from selected channels and sets it on active user; GET /api/notifications/badge returns count; POST /api/notifications/reset resets badge.
Frontend	index.html + app.js	Added Console/Email/SMS/Dashboard checkboxes; badge bubble next to heading; updateChannels() sends POST on checkbox change; fetchBadgeCount() polls every 5s; resetBadge() on button click.

Change 3 — Multiple Users (Singleton + Observer Pattern)

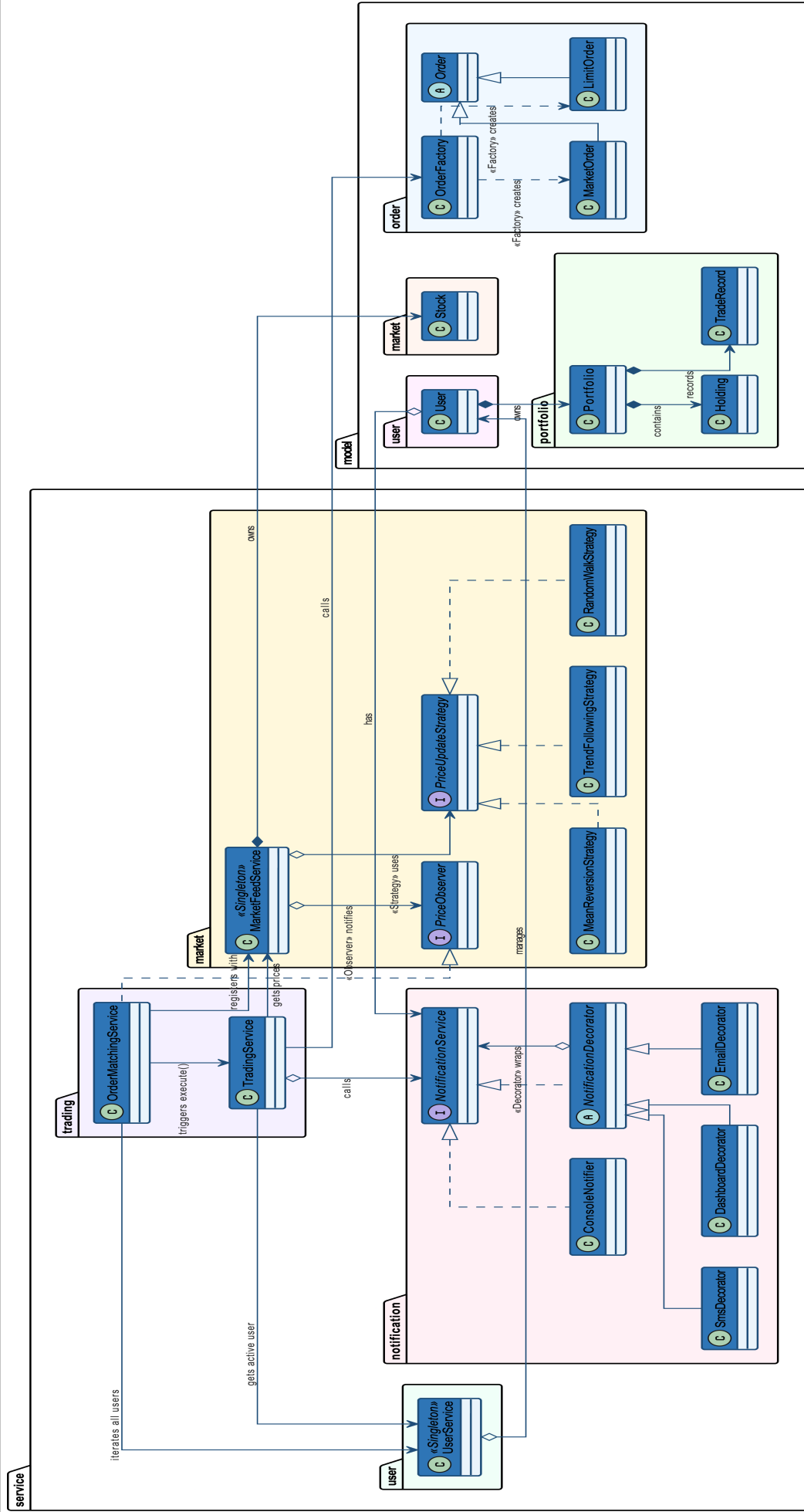
What changed	File	Exactly what was modified / added
Modified	Portfolio.java	Removed @Component — Portfolio is no longer a global singleton; added List pendingOrders field with getPendingOrders() so each user owns their own pending order list.
Modified	TradingService.java	Replaced Portfolio + NotificationService constructor params with UserService; getPortfolio() and getNotificationService() now delegate to userService.getActiveUser() on every call.
Modified	OrderMatchingService.java	Injected UserService; onPriceUpdate() now iterates ALL users, checks each user's pending orders, temporarily sets active user to execute their orders, then restores previous active user.
Modified	PortfolioController.java	Replaced Portfolio injection with UserService; getPortfolio() now calls userService.getActiveUser().getPortfolio() so response always reflects the currently active user.
Modified	TradeController.java	Replaced Portfolio injection with UserService; getTradeHistory() now returns userService.getActiveUser().getPortfolio().getTradeHistory() for per-user trade history.
Modified	NotificationController.java	Injected UserService; setChannels() now calls userService.getActiveUser().setNotificationService(chain) so notification preferences are stored per user not globally.

Modified	TradingServiceTest.java	Updated constructor to take UserService + MarketFeedServices; replaced Portfolio assertions with userService.getActiveUser().getPortfolio(); added users_have_isolated_portfolios test.
Modified	OrderMatchingServiceTest.java	Updated constructors to inject UserService; updated assertions to read portfolio from userService.getActiveUser().getPortfolio().
New	User.java	Holds id, name, Portfolio instance, and NotificationService per user; each User is created with a fresh Portfolio (\$10,000 cash) and a ConsoleNotifier.
New	UserService.java	@Service singleton; creates Alice, Bob, Charlie at startup in a LinkedHashMap; getActiveUser(), setActiveUser(), getUser(), getAllUsers() — throws IllegalArgumentException for unknown user.
New	UserController.java	GET /api/users lists all users; GET /api/users/active returns active user; POST /api/users/active switches active user and triggers full UI refresh.
New	UserServiceTest.java	8 tests: default user is Alice, switch to Bob/Charlie, unknown user throws, 3 users exist, each has independent portfolio, all start with \$10,000, switching does not affect others.
Frontend	index.html + app.js	Added user selector dropdown at top of page; fetchActiveUser() syncs dropdown on load; switchUser() sends POST /api/users/active and calls refreshAll() to reload portfolio for new user.

Summary — Files Modified and Added

Change	Pre-existing files modified	New files added	Tests
Change 1 Pricing Algorithms	5 Java files (PriceUpdateStrategies, RandomWalkStrategy, MarketFeedServices, AppConfig, RandomWalkStrategyTest)	5 files (MeanReversionStrategy, TrendFollowingStrategy, StrategyController, 2 test files)	43 total (+10 new)
Change 2 Notifications	5 Java files (NotificationDecorator, TradingService, ConsoleNotifier, AppConfig, NotificationServiceTest)	4 files (EmailDecorator, SmsDecorator, DashboardDecorator, NotificationController)	43 total (+5 new chain tests)
Change 3 Multiple Users	8 Java files (Portfolio, TradingService, OrderMatchingService, PortfolioController, TradeController, NotificationController, TradingServiceTest, OrderMatchingServiceTest)	4 files (User, UserService, UserController, UserServiceTest)	43 total (+8 user tests)

Blue = pre-existing file modified	Green = new file added	Yellow = frontend (index.html + app.js)



Symbol	Meaning	Inheritance
<	--	Interface implement
<	..	
o-->	Aggregation	
*-->	Composition	
-->	Dependency	
..>	Creates / uses	

